Help Menu

The commands are described under the **Help Menu**.

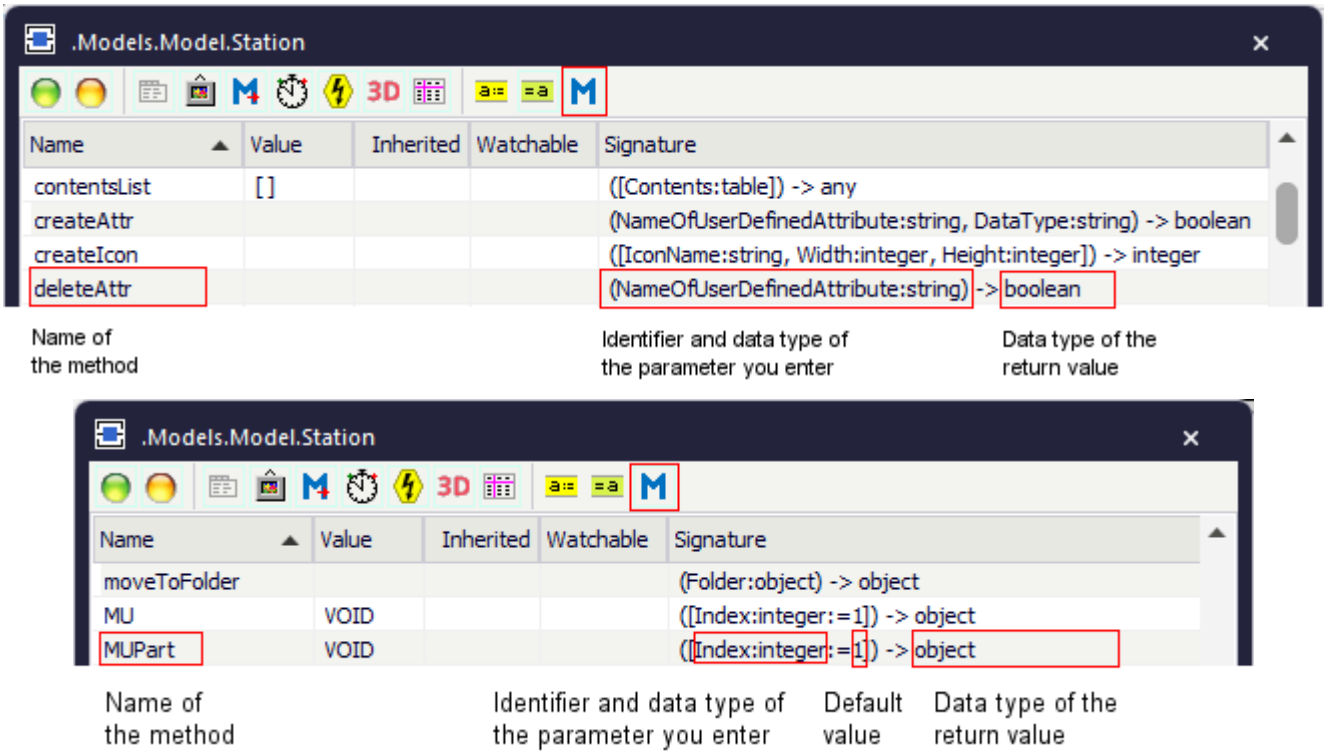
Methods of the Chart

_Methods of the Chart

The *Chart* provides:

- The *methods* listed in the table of contents to the left.
- The **Methods of All Objects**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class [general description]**.
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance [general description]**.

An example of the **Syntax** line of the individual methods might look like this:

```
<Path>.openDialog([CallOpenControl:boolean:=false]) → boolean
```

- The expression *<Path>* designates the path of the object to which the *method* applies.
- The signature of the method, consisting of the identifier and the data type of the parameter, is listed in parentheses. The expression *(Parameter:string)*, for example, designates a parameter of data type *string*. Instead of a constant value, you can also use a variable of the required type or a *method* that returns the required data type.

  Note

  Make sure to enter the parentheses for expressions within parentheses (...). Not entering them may lead to unexpected results and open the *Debugger*.

  Optional parameters are listed within brackets. The expression *[,Parameter:boolean]*, for example, means that you can, but do not have to enter the *boolean* parameter.

  If a parameter has a default value, the signature shows the default value after the parameter, `:= false` in the example above.

  If the *method* has a return value, the signature shows its data type after the arrow ->, `→ boolean` in the example above.

addObject [SimTalk] - Chart

Adds the designated object to the *Chart* designated by <Path>.

Remarks

This works the same way as if you dragged the object to the *Chart* and dropped it there.

Type

Method

Syntax

```
<Path>.addObject(Object:object) → boolean
```

Parameter

The parameter *Object* of data type *object* designates the name of the object.

Return Value

The return value has the data type *boolean*.

Example

```
MyChart.addObject(MyExporter)
```

See also

Drag-and-Drop in the Chart

copyBitmapToClipboard [SimTalk] - Chart

Copies the contents of the display window of the *Chart* designated by <Path> to the clipboard as a bitmap.

Type

Method

Syntax

```
<Path>.copyBitmapToClipboard([Width:integer, Height:integer])
```

Parameters

You can specify the following parameters:

- The optional parameter *Width* of data type *integer* designates the width of the bitmap in pixels.
- The optional parameter *Height* of data type *integer* designates its height.

Example

```
MyChart.copyBitmapToClipboard
MyChart.copyBitmapToClipboard(180,120)
```

SimTalk

copyBitmapToIcon [SimTalk]

See also

Copy to Clipboard

copyBitmapToFile [SimTalk] - Chart

Copies the graphic of the window of the *Chart* designated by <Path> to a file as a .png file.

Type

Method

Syntax

```
<Path>.copyBitmapToFile(FileName:string, Width:integer, Height:integer)
```

Parameters

You can specify the following parameters:

- The parameter *FileName* of data type *string* designates the name of the file to which *Plant Simulation* writes the .png file.
- The parameter *Width* of data type *integer* designates the width of the bitmap.
- The parameter *Height* of data type *integer* designates the height of the bitmap.

Example

```
.Models.MyChart.copyBitmapToFile("myChartImage",180,120)
```

SimTalk

copyBitmapToClipboard [SimTalk] - Chart

copyBitmapToIcon [SimTalk]

copyBitmapToIcon [SimTalk]

Copies the graphic of the window of the *Chart* designated by <Path> to an icon of the designated object.

Remarks

copyBitmapToIcon applies to the icon of the addressed instance not to the icon of the object class.

Type

Method

Syntax

```
<Path>.copyBitmapToIcon(Destination:object, IconNumberOrName:integer/
string[ ,Width:integer, Height:integer])
```

Parameters

You can specify the following parameters:

- The parameter *Destination* of data type *object* designates the object into which the bitmap is to be copied.
- The parameter *IconNumberOrName* of data type *integer* or *string* designates the icon. If the object does not have an icon with the name or the number respectively yet, *Plant Simulation* creates a new icon.
- The optional parameter *Width* of data type *integer* designates the width of the bitmap.
- The optional parameter *Height* of data type *integer* designates the height of the bitmap.

  If you do not specify the optional parameters, *Plant Simulation* uses the current width and height of the *Chart*.

Example

```
MyChart.copyBitmapToIcon(Station1, -1)
```

SimTalk

copyBitmapToClipboard [SimTalk] - Chart

copyBitmapToFile [SimTalk] - Chart

getAnnotations [SimTalk]

Returns the contents of the table **Annotations** of the *Chart* designated by <Path>.

Type

Method

Syntax

```
<Path>.getAnnotations(AnnotationsToGet:table)
```

Parameter

The parameter *AnnotationsToGet* of data type *table* designates the table into which the annotations table is going to be written.

Example

```
MyChart.getAnnotations(MyAnnotationsTable)
```

SimTalk

setAnnotations [SimTalk]

See also

Add Labels and a Legend

Annotations [button]

getColor [SimTalk]

Returns the RGB value of the **Color** of an item in the *Chart* designated by <Path>.

Type

Method

Syntax

```
<Path>.getColor(ColorNo:integer[, byref Opacity:integer]) -> integer
```

Parameters

You can specify the following parameters:

- The parameter *ColorNo* of data type *integer* designates the color of the element.
  - **Grid** (number **-3**)

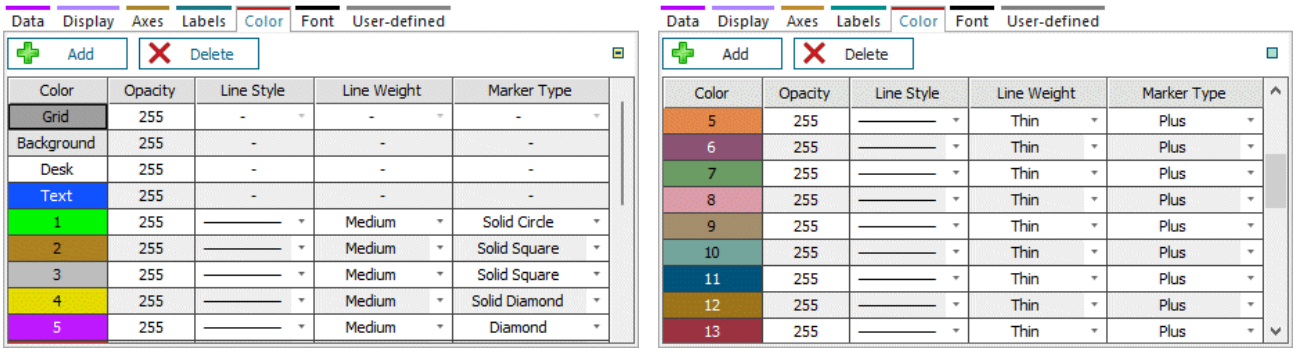
    Is the color the *Chart* uses for displaying the grid, i.e., the bounding rectangles, and the grid lines of the graph proper in the *Chart* window.
  - **Background** (number **-2**)

    Is the color the *Chart* uses for displaying the background of the graph proper in the *Chart* window.
  - **Desk** (number **-1**)

    Is the color the *Chart* uses for displaying the area located outside of the bounding rectangle of the graph's grid in the *Chart* window.
  - **Text** (number **0**)

    Is the color the *Chart* uses for displaying text.
  - **Colors** (numbers **1** through **14**)

    1 through **14** are the predefined colors of the input channels of the displayed data.



- The optional parameter *Opacity* of data type *integer* designates the opacity of the color.

Return Value

The return value has the data type *integer*.

Example

```
var rgb: integer := Chart.getColor(3)  -- reads the RGB color value of
color 3
var opacity: integer
Chart2.setColor(3, Chart1.getColor(3, opacity), opacity)
-- copies the color value and the opacity of color 3 of Chart1 to Chart2
```

SimTalk

setColor [SimTalk]

See also

Color [Chart]

getHTMLCode [SimTalk] - Chart

Returns the chart graphic as HTML code in the format SVG of the *Chart* designated by <Path>.

Type

Method

Syntax

```
<Path>.getHTMLCode([Caption:string, Width:integer, Height:integer,
inMM:boolean]) → string
```

Parameters

You can specify the following parameters:

- The optional parameter *Caption* of data type *string* designates the desired caption of the graphic. Specify the caption to show the chart with this caption.
- The optional parameter *Width* of data type *integer* designates the desired **Width** of the graphic.
- The optional parameter *Height* of data type *integer* designates the desired **Height** of the graphic.

  If you do not specify the width and the height, *Plant Simulation* uses the width and the height of the *Chart* window.
- The optional parameter *inMM* of data type *boolean* specifies if the width and the height of the graphic was specified in millimeters (true) or in pixels (false).

Return Value

The return value has the data type *string*.

Example

```
print MyChart.getHTMLCode
```

See also

Display a HtmlReport

getLineStyle [SimTalk]

Returns the settings of the line style in the *Chart* designated by <Path> and writes them into the passed parameters.

Type

Method

Syntax

```
<Path>.getLineStyle(ColorNumber:integer, byRef Style:string, byRef
Weight:string, byRef Marker:string)
```

Parameters

The parameter *ColorNumber* of data type *integer* designates the **Color**.

- **Grid** (number **-3**)

  Is the color the *Chart* uses for displaying the grid, i.e., the bounding rectangles, and the grid lines of the graph proper in the *Chart* window.

- **Background** (number **-2**

  Is the color the *Chart* uses for displaying the background of the graph proper in the *Chart* window.

- **Desk** (number **-1**)

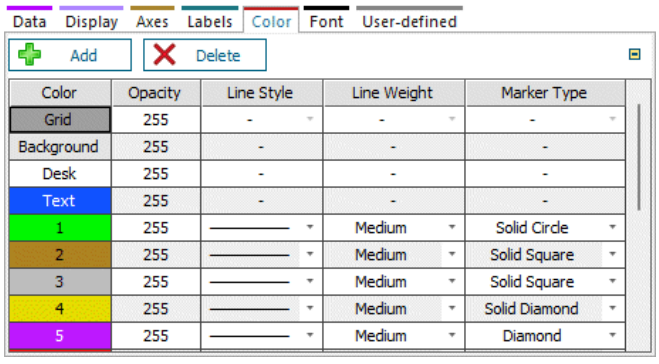
  Is the color the *Chart* uses for displaying the area located outside the bounding rectangle of the graph's grid in the *Chart* window.

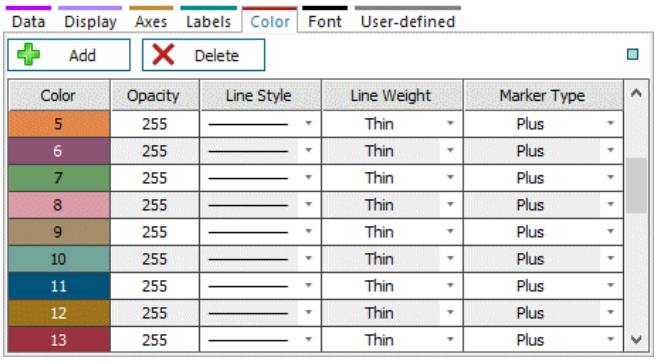
- **Text** (number **0**)

  Is the color the *Chart* uses for displaying text.

- **Colors** (numbers **1** through **14**)

  1 through **14** are the predefined colors of the input channels of the displayed data.





- The local variable *Style* of data type *string*, to which the value is assigned, designates the **Line Style**.
- The local variable *Weight* of data type *string*, to which the value is assigned, designates the **Line Weight**.
- The local variable *Marker* of data type *string*, to which the value is assigned, designates the **Marker Type**.

Example

```
var s,w,m : string
MyChart.getLineStyle(1,s,w,m)
print s," ",w," ",m
```

SimTalk

setLineStyle [SimTalk]

See also

Color [Chart]

printChart [SimTalk]

Prints the contents of the window of the *Chart* designated by <Path> to the default printer.

Type

Method

Syntax

```
<Path>.printChart([Width:real, Height:real, Orientation:integer])
```

Parameters

You can specify the following parameters:

- The optional parameter *Width* of data type *real* designates the width of the printed *Chart* in millimeters.
- The optional parameter *Height* of data type *real* designates the height of the printed *Chart* in millimeters.
- The optional parameter *Orientation* of data type *integer* designates the orientation of the printed *Chart*.

  You can specify `0` for the printer **Default**, `1` for **Landscape**, or `2` for **Portrait**.

Example

```
MyChart.printChart
MyChart.printChart(160,150,2)
```

See also

Print [Chart]

putValuesIntoTable [SimTalk] - Chart

Writes the collected values of the *Chart* designated by <Path> into the designated table.

Remarks

putValuesIntoTable is especially useful if you selected the **Category > Histogram** or the **Category > Plotter** as then the *Chart* collects values over time which you cannot easily access in any other way.

Type

Method

Syntax

```
<Path>.putValuesIntoTable(DestinationTable:table)
```

Parameter

The parameter *DestinationTable* of data type *table* designates the table.

Example

```
MyChart.putValuesIntoTable(MyDataTable)
```

See also

Category [drop-down list]

resetValues [SimTalk]

Resets the collected values for the **Category > Histogram** and the **Category > Plotter** of the *Chart* designated by <Path>.

Remarks

resetValues deletes the existing data and starts collecting data anew.

You can, for example, use it to record a weekly histogram or to suppress data collected during the ramp-up phase.

Type

Method

Syntax

```
<Path>.resetValues
```

Example

```
MyChart.resetValues
```

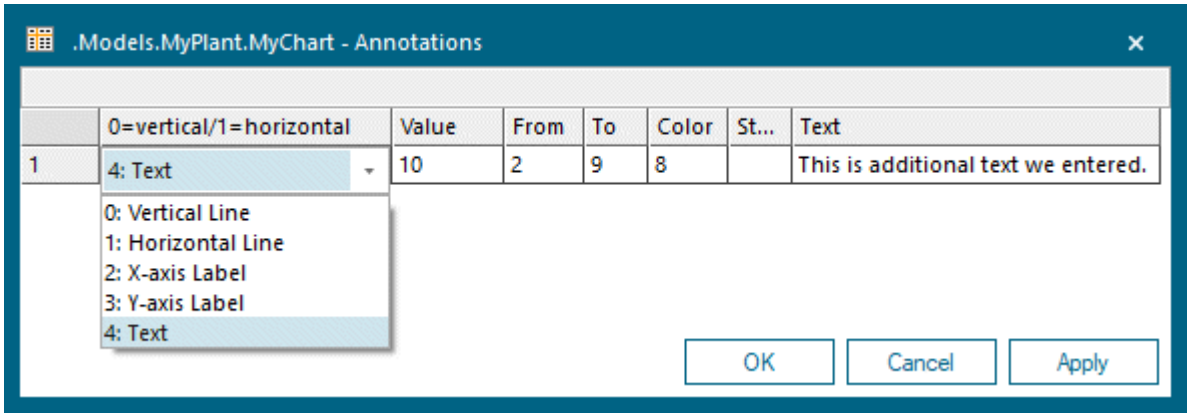
See also

Copy to Clipboard

Category [drop-down list]

setAnnotations [SimTalk]

Sets the table **Annotations** of the *Chart* designated by <Path>.



Remarks

Plant Simulation always shows user-defined annotations in the foreground. In previous versions this was not the case if the **Y-Axis** showed time values. As the Chart component, which *Plant Simulation* uses, cannot display times, *Plant Simulation* creates the captions for time values on the **Y-Axis** and the horizontal grid lines as **annotations**.

If you do add your own **annotations** to such a *Chart*, *Plant Simulation* now shows all **annotations**, including the grid lines, in the foreground. In this case, we recommend to hide the grid lines or to set the **Color** of the grid to a light color, for example to light gray.

Type

Method

Syntax

```
<Path>.setAnnotations(AnnotationsToSet:table)
```

Parameter

The parameter *AnnotationsToSet* of data type *table* designates the table that contains the contents of the annotations table.

By entering the following information into the table, you can add additional lines and text to the *Chart*:

- **Type**

  Select the **Type** of the annotation:

Select `0: Vertical Line` to show a vertical line.

Select `1: Horizonal Line` to show a horizontal line.

Select `2: X-axis Label` for labeling the x-axis.

Select `3: Y-axis Label` for labeling the y-axis.

Select `4: Text` for any text.

- **Value**

  Enter the value where the *Chart* displays the line in the *Chart* window.

  For a horizontal line or for an annotation of the Y axis or for this is the Y value.

  For a vertical line or for an annotation of the X axis this is X value.

  Enter Y values in the Y scale of the *Chart*.

  Enter X values in the X scale of the *Chart*. For a bar chart the value 3.5 means that the X value is placed between the third and the fourth bar.

- **From**

  Enter the starting point of the line here. If you do not enter a value, the *Chart* extends an existing line.

  This way you can not only create horizontal and vertical lines but also diagonal lines.

  For a horizontal line or for text this is an X value. For a vertical line this is a Y value.

- **To**

  Enter the end point of the line here.

  If you do not enter a value into the cells **From** and **To**, the *Chart* draws the line from the left to right, or from the top to the bottom of the display window.

  For a horizontal line this is a X value. For a vertical line this is a Y value.

- **Color**

  Enter the number of the color of the line here. Define colors on the **Tab Color**.

- **Style**

  For lines of **Type** `0` or `1`, you can enter the line style.

  For text of **Type** `4`, you can enter the marker style.

Value	Line style	Marker style
`0`	a thin solid line	text only without a marker
`1`	a dashed line	a plus sign
`2`	a dotted line	a cross
`3`	a dash-dot line	a circle
`4`	a dash-dot-dot line	a filled circle

Value	Line style	Marker style
5	a medium thin solid line	a square
6	a thick solid line	a filled square
7	a grid tick	a diamond
8	a grid line	a filled diamond
9	none	upward triangle
10	a medium thick solid line	solid upward triangle
11	an extra thick solid line	downward triangle
12	none	solid downward triangle
13	none	small plus
14	none	small cross
15	none	small circle
16	none	small solid circle
17	none	small square
18	none	small solid square
19	none	small diamond
20	none	small solid diamond
21	none	small upward triangle
22	none	small solid upward triangle
23	none	small downward triangle
24	none	small solid downward triangle
25	none	large plus
26	none	large cross
27	none	large circle
28	none	large solid circle
29	none	large square
30	none	large solid square
31	none	large diamond
32	none	large solid diamond
33	none	large upward triangle

Value	Line style	Marker style
34	none	large solid upward triangle
35	none	large downward triangle
36	none	large solid downward triangle
92	none	arrow north
93	none	arrow north east
94	none	arrow east
95	none	arrow south east
96	none	arrow south
97	none	arrow south west
98	none	arrow west
99	none	arrow north west

- **Text**

  Enter any text that describes the line here. You can optionally enter the following two-character prefix codes, consisting of the pipe symbol (|) and a letter. These control at which position in the graph the text will be positioned:

Prefix	Position
|l	left inside edge of the graph
|L	left outside edge of the graph
|r	right inside edge of the graph
|R	right outside edge of the graph
|c	centered inside of the graph

Examples

```
MyChart.Annotations := setAnnotations(myAnnotationTable)
```

```
-- Generation of a box plot for data with the discriptive statistics in
tablefile 'StatisticData'
var AnnotationsTab:table;var Q0,Q25,Q50,Q75,Q100, yScaleMax,
yScaleMin,delta, Mu:real; var cNo:integer;var ValName:string

Chart.active := false

AnnotationsTab := Chart.Annotations; AnnotationsTab.delete
```

```
yScaleMin := 1E300 -- all parts of the box plot must be visible
yScaleMax := 0
delta := 0.1 -- width of the column for the interval from the lower to the
upper quartile (must be positive and < 0.5)
cNo := 3 -- blue, color for the intervals
Chart.title := "Box Plot"
for var ValNo := 1 to StatisticalData.xDim
    ValName := StatisticalData[ValNo, 0]
    Q0   := StatisticalData[ValNo, "Minimum"]
    Q25  := StatisticalData[ValNo, "Lower Quartile"]
    Q50  := StatisticalData[ValNo, "Median"]
    Q75  := StatisticalData[ValNo, "Upper Quartile"]
    Q100 := StatisticalData[ValNo, "Maximum"]
    Mu   := StatisticalData[ValNo, "Mean Value"]
    if Q0>Q25 OR Q25>Q50 OR Q50>Q75 OR Q75>Q100 OR Mu>Q100 OR Mu<Q0
        Promptmessage(to_str("Invalid statistical data ", when
strLen(ValName) < 1 then to_str("in column ",valNo) else to_str("for
'",ValName,"'"),"."))
    end
    yScaleMax := max(yScaleMax,Q100)
    yScaleMin := min(yScaleMin,Q0)

    AnnotationsTab.writeRow(1, AnnotationsTab.ydim + 1, 0,
ValNo,Q25,Q0,cNo,0 )
    AnnotationsTab.writeRow(1, AnnotationsTab.ydim + 1, 0,
ValNo,Q100,Q75,cNo,0 )
    AnnotationsTab.writeRow(1, AnnotationsTab.ydim + 1, 0, ValNo +
delta,Q25,Q75,cNo,0 )
    AnnotationsTab.writeRow(1, AnnotationsTab.ydim + 1, 0, ValNo -
delta,Q25,Q75,cNo,0 )

    AnnotationsTab.writeRow(1, AnnotationsTab.ydim + 1, 1, Q25, ValNo -
delta, ValNo + delta,cNo,0 )
    AnnotationsTab.writeRow(1, AnnotationsTab.ydim + 1, 1, Q50, ValNo -
delta, ValNo + delta,cNo,0 )
    AnnotationsTab.writeRow(1, AnnotationsTab.ydim + 1, 1, Q75, ValNo -
delta, ValNo + delta,cNo,0 )

    AnnotationsTab.writeRow(1, AnnotationsTab.ydim + 1, 1, Q0, ValNo -
delta, ValNo + delta,cNo,0 )
    AnnotationsTab.writeRow(1, AnnotationsTab.ydim + 1, 1, Q100, ValNo -
delta, ValNo + delta,cNo,0 )
next

delta := (Q100 - Q0)/10
Chart.yScaleMax := Q100 + delta
Chart.yScaleMin := Q0 - delta

Chart.active := true
```

SimTalk

getAnnotations [SimTalk]

See also

Annotations [button]

Add Labels and a Legend

setColor [SimTalk]

Sets the **Color** of an item of the *Chart* designated by <Path>.

Type

Method

Syntax

```
<Path>.setColor(ColorNo:integer, RGB:integer[, Opacity:integer])
```

Parameters

You can specify the following parameters:

- The parameter *ColorNumber* of data type *integer* designates the color of the item.
  - **Grid** (number -3)

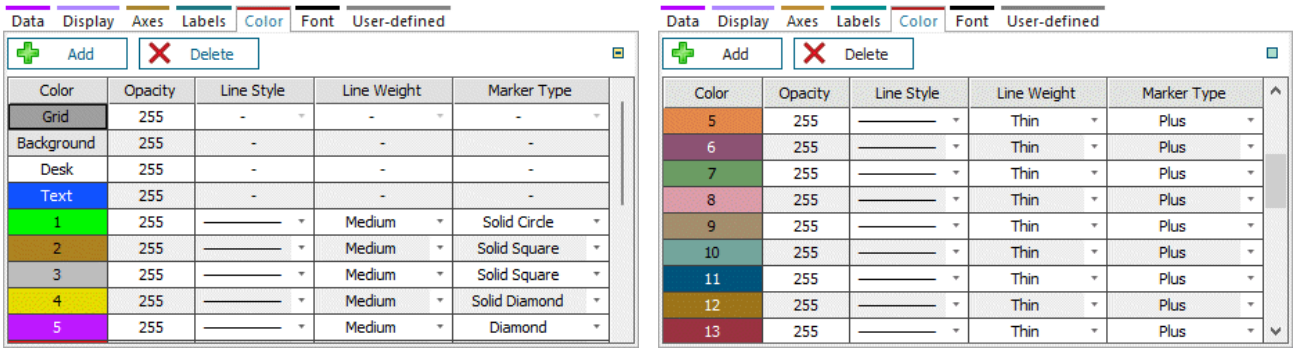
    Is the color the *Chart* uses for displaying the grid, i.e., the bounding rectangles, and the grid lines of the graph proper in the *Chart* window.
  - **Background** (number -2)

    Is the color the *Chart* uses for displaying the background of the graph proper in the *Chart* window.
  - **Desk** (number -1)

    Is the color the *Chart* uses for displaying the area located outside the bounding rectangle of the graph's grid in the *Chart* window.
  - **Text** (number 0)

    Is the color the *Chart* uses for displaying text.
  - **Colors** (numbers 1 through 14)

    1 through **14** are the predefined colors of the input channels of the displayed data.



- The parameter *RGB* of data type *integer* designates the red, green, and blue component of this color.
- The optional parameter *Opacity* of data type *integer* designates the opacity of the color.

  Enter `0` for an entirely transparent color, enter `255` for a completely opaque color. Semi-transparent colors will, for example, increase the readability of the **Chart Type** > **Area**.

Example

```
MyChart.setColor(1, makeRGBValue(255,128,0))      -- sets the first color
to orange
MyChart.setColor(2, makeRGBValue(255,0,0), 127))  -- sets the second color
to a semi-transparent red
MyChart.setColor(-2, makeRGBValue(255,255,0))     -- sets the background
color to yellow
```

SimTalk

getColor [SimTalk]

See also

Color [Chart]

setLineStyle [SimTalk]

Sets the settings of the line style of the *Chart* designated by <Path>.

Type

Method

Syntax

```
<Path>.setLineStyle(ColorNumber:integer, LineStyle:string[,
LineWeight:string, Marker:string])
```

Parameters

You can specify the following parameters:

- The parameter *ColorNumber* of data type *integer* designates the **Color**.
  - **Grid** (number **-3**)

    Is the color the *Chart* uses for displaying the grid, i.e., the bounding rectangles, and the grid lines of the graph proper in the *Chart* window.
  - **Background** (number **-2**)

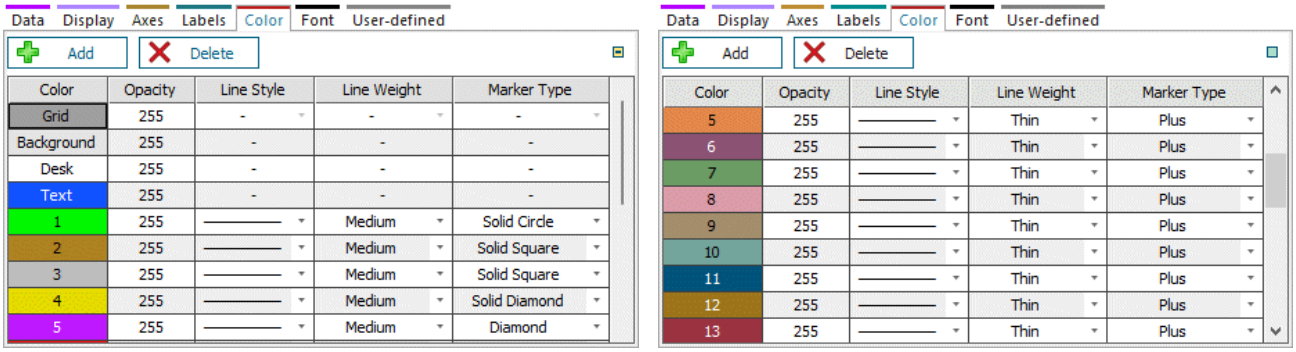
    Is the color the *Chart* uses for displaying the background of the graph proper in the *Chart* window.
  - **Desk** (number **-1**)

    Is the color the *Chart* uses for displaying the area located outside the bounding rectangle of the graph's grid in the *Chart* window.
  - **Text** (number **0**)

    Is the color the *Chart* uses for displaying text.
  - **Colors** (numbers **1** through **14**)

    1 through **14** are the predefined colors of the input channels of the displayed data.

    You can also **add** additional colors and define their properties to meet your modeling needs.

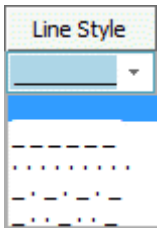


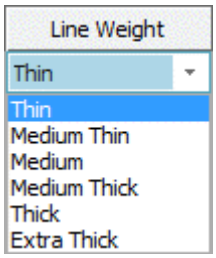
- The parameter *LineStyle* of data type *string* designates the **Line Style**.

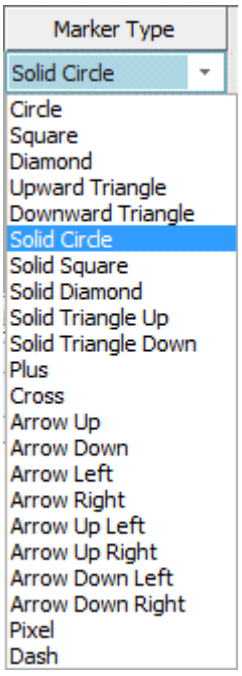
  You can also specify a shortened *string* to the method, which has to have a length of least three characters. You can, for example, specify ". ." or ". . ." instead of ". . . . . ." to create a dotted line. You can also specify "_ _" instead of "_ _ _ _ _ _" to create a dashed line.

  You can also specify minus signs instead of underscores, for example "- -" instead of underscores "_ _".

- The optional parameter *LineWeight* of data type *string* designates the **Line Weight**. If you do not specify it, *Plant Simulation* keeps the previous *LineWeight*.
- The optional parameter *Marker* of data type *string* designates the **Marker Type**.







Example

```
MyChart.setLineStyle(1, "--", "Thick", "Diamond")
```

See also

getLineStyle [SimTalk]

setWindowPosition [SimTalk] - Chart

Sets the position of the window of the *Chart* designated by <Path> and its width and height.

Type

Method

Syntax

```
<Path>.setWindowPosition(X:integer, Y:integer, Width:integer,
Height:integer)
```

Parameters

You can specify the following parameters:

- The parameter *X* of data type *integer* designates the x-coordinate of the window.
- The parameter *Y* of data type *integer* designates its y-coordinate.
- The parameter *Width* of data type *integer* designates its width.
- The parameter *Height* of data type *integer* designates its height.

Example

```
MyChart.setWindowPosition(200,100,240,250)
```

showPrintDialog [SimTalk] - Chart

Opens the dialog **Print** of the *Chart* designated by <Path>.

Type

Method

Syntax

```
<Path>.showPrintDialog([Width:real, Height:real])
```

Parameters

You can specify the following parameters:

- The optional parameter *Width* of data type *real* designates the width of the printed *Chart* in millimeters.
- The optional parameter *Height* of data type *real* designates the height of the printed *Chart* in millimeters.

Example

```
MyChart.showPrintDialog
MyChart.showPrintDialog(140,120)
```

See also

Print [Chart]

update [SimTalk] - Chart

Refreshes the displayed *Chart* designated by <Path> with the current values.

Remarks

If the *Chart* is a *Plotter* in **Sample** mode, the method *update* has an additional effect, namely that a new data point is set, i.e., the *Chart* samples one more time, even then when the *Chart* is closed and the method returns false for this reason.

Type

Method

Syntax

```
<Path>.update -> boolean
```

Return Value

The return value has the data type *boolean*.

true if the *Chart* is open.

false if it is closed.

Example

```
MyChart.update
```

See also

Mode [drop-down list] - Chart

Read-Only Attributes of the Chart

Read-Only Attributes of the Chart

The *Chart* provides the **_Read-Only Attributes of All Objects**.

You can query the values of the *read-only attributes*, but you cannot set them as *Plant Simulation* computes the value for the point-in-time at which you query it. In most cases a *read-only attribute* corresponds to an unavailable dialog item on one of the tabs of the object, for example on the tab **Statistics**.